协同验证与性能分析

以 NEMU 和标准 RVV 基线建立正确性基础，先测量完整操作延迟，再分析含访存任务与同输出优化对照。

03 / 验证与性能 | 赛题 3.1.1、3.1.2、3.2、3.3
实验材料截至 2026-10-30 · 2026-11-02 第三版修订

目录

7 NEMU 扩展与差分验证

7.1 NEMU 与 RTL 共用 v0.1 指令规格
7.2 custom-0 译码规则与 def_EHelper(vdot)
7.3 仿真器、参考库与测试镜像的构建和加载
7.4 API 268 例与整核四类套件
7.5 mtval 失配：现象、原因、改法与回归
7.6 OVERLAP 与 PIPELINE 的运行记录

8 标准 RVV 等价基线

8.1 kernel_end 比较点与寄存器约定
8.2 vwmul.vv 加 vwredsum.vs：两次加宽得到 INT32
8.3 三条核心段：1 条对 9 条静态指令
8.4 等价验证用例：6 组数据、144 种配置、432 次调用
8.5 整核运行通过，GOOD TRAP 位于 0x80000858

9 延迟测量与软件优化分析

9.1 用依赖链增量斜率定义延迟
9.2 CSRRW 写 mscratch 作计时边界
9.3 依赖链延迟：7 对 55 / 47 周期
9.4 八路批处理：VDot 增量 1 周期/次
9.5 逐点输出小数组：N=128 时基本持平
9.6 对照A：分段归约省去中间落存
9.7 对照B：调整输入策略和加载安排
9.8 对照C：2×2 小块复用输入
9.9 性能结论与适用条件

附录 C 模型与性能索引

7 NEMU 扩展与差分验证

参考模型和整核差分全部通过：

- 参考模型 API 检查：268 例通过（另有 10 例配置不适用，单列，不计入通过数）。
- 修复版整核差分套件：SMOKE 7 例、EXCEPTION 64 例、OVERLAP 60 例、PIPELINE 120 例，全部通过。
- 回归中发现的短 VL 异常 mtval 失配已在 RTL 中修复，原异常用例回归通过，见 7.5。

自定义指令接入 RTL 后，参考模型也必须识别同一机器码、实现相同的架构语义。我们在源码 NEMU 中增加 vdot.vv，让测试应用、RTL 与 NEMU 在同一运行中互相校验：应用对照独立的数学期望，DiffTest 对照两侧的架构状态，波形解释内部事件。三个角度各有分工，只看最终数值容易漏掉异常或状态差异。

本章依次说明参考模型实现、验证范围和短 VL 异常中的 mtval 失配。这个案例展示了差分检查怎样从一个 CSR 寄存器值的差异出发，找到整核中漏传指令字的环节。

7.1 NEMU 与 RTL 共用 v0.1 指令规格

参考模型与 RTL 遵循同一份项目 v0.1 指令规格，合法配置范围和状态更新规则一致〔1〕。

在向量执行权限可用的前提下，合法配置要求 vill=0、SEW=8、LMUL=1、vstart=0，VL 为 0 或 8～16，vtype 保留高位为零。合法非零 VL 固定计算源低八项，精确和写入低 32 位、高 96 位清零；合法 VL=0 保持完整旧目的。类型、权限和重启条件的检查排在长度分支之前。

RTL 基于 XiangShan 16ae9ddc，叠加 VDot 和异常修复；参考模型基于 NEMU a48a11f2，叠加独立的语义实现。完整版本和二进制文件校验值见阶段三复现索引。

7.2 custom-0 译码规则与 def_EHelper(vdot)

NEMU 用一条 custom-0 译码规则生成 vdot 执行 ID，最终调用 `def_EHelper(vdot)`。

NEMU 修改分为四部分：译码入口、执行 ID 注册、向量计算头文件接入和新增语义文件。custom-0 的末七位为 `0001011`，funct6、funct3 均为零，其余字段保留 vm 与寄存器编号。实际译码规则为：

```
#ifdef CONFIG_RVV
  def_INSTR_IDTAB("000000 ? ????? ????? 000 ????? 00010 11", OP_V  , vdot);
  def_INSTR_IDTAB("??????? ????? ????? ??? ????? 10101 ??", OP_V  , OP_V);
#endif // CONFIG_RVV
```

第一条规则识别 custom-0 位型并生成 vdot 执行 ID，第二条继续处理原有的标准 OP-V。执行 ID 注册到 isa-all-instr.h，vcompute.h 包含新增的 vdot.h，最终调用 `def_EHelper(vdot)`。机器码识别与计算语义由此有了明确的连接路径。

执行函数如下，摘自 vdot.h 源码〔2〕：

```
def_EHelper(vdot) {
  require_vector(true); // host/guest VS and vill, before the vl=0 path
  if ((vtype->val >> 8) != 0 || vtype->vsew != 0 || vtype->vlmul != 0 ||
      vstart->val != 0 || (vl->val != 0 && (vl->val < 8 || vl->val > 16))) {
    longjmp_exception(EX_II);
  }
  const uint32_t word = s->isa.instr.val;
  const unsigned vd = (word >> 7) & 31;
  const unsigned vs1 = (word >> 15) & 31;
  const unsigned vs2 = (word >> 20) & 31;
  const unsigned vm = (word >> 25) & 1;
  if (vl->val != 0) {
    // Snapshot before any write, including vd=v0 and all source overlaps.
    const uint64_t a = cpu.vr[vs1]._64[0];
    const uint64_t b = cpu.vr[vs2]._64[0];
    const uint64_t mask = cpu.vr[0]._64[0];
    int32_t sum = 0;
    for (unsigned i = 0; i < 8; ++i) {
      const int ai = ((int)((a >> (8*i)) & 255) ^ 128) - 128;
      const int bi = ((int)((b >> (8*i)) & 255) ^ 128) - 128;
      if (vm || ((mask >> i) & 1)) sum += ai * bi;
    }
    cpu.vr[vd]._64[0] = (uint32_t)sum;
    cpu.vr[vd]._64[1] = 0;
  }
  vstart->val = 0;
  vp_set_dirty(); // includes legal vl=0 and guest VS when virtualized
}
```

实现中有三处与边界测试直接相关：

- 合法性检查位于 VL=0 之前，空操作也不能绕过异常条件。
- 两个源和掩码先存入局部变量，再改写目的；vd 与源或 v0 重叠时仍读取旧值。
- `(uint32_t)sum` 写入低 64 位容器时高 32 位为零，另一个 64 位字也置零，恰好形成低 32 位结果与高 96 位零。

合法 VL=0 跳过目的更新，但仍执行成功路径的状态处理；`vp_set_dirty()` 复用 NEMU 已有的向量状态机制。源文件及译码入口统一保存在 NEMU 修改快照中〔3〕。

7.3 仿真器、参考库与测试镜像的构建和加载

协同仿真需要分别编译 RTL 仿真器 `emu`、NEMU 参考库 `riscv64-nemu-interpreter-so` 和 RISC-V 测试镜像。运行时，`emu` 加载测试镜像，通过 `--diff` 指定 NEMU 参考库，通过 `-F` 指定 Flash 镜像。



图 7-1　验证结构。应用对照独立期望值；DiffTest 在退休、异常等同步点比较 RTL 与 NEMU 导出的架构状态。

DiffTest 经 regcpy 交换状态。按本配置两侧的源码布局，交换区为 1400 字节；向量区从偏移 800 字节开始，32 个 128 位向量寄存器共 512 字节。参考库单步检查直接注入并回读这些状态，整核联合运行再检验实际的同步和执行行为。布局依据见接口核对记录〔4〕。

验收同时查看参考库路径、DiffTest 启用、应用 PASS、GOOD TRAP 和被测程序退出码；脚本及日志收集的退出状态单列记录。完整构建与运行命令入口见交付与复现记录〔5〕。

7.4 API 268 例与整核四类套件

API 层 268 例通过、10 例不适用单列；整核层 SMOKE、EXCEPTION、OVERLAP 和 PIPELINE 四类套件全部通过。

参考模型先经 API 接口做状态注入、单条执行和回读。检查覆盖完整的 32 个向量寄存器及相关 CSR，结果如下：

表 7-1　参考模型 API 检查

类别	数量	主要检查
合法非零 VL	93	目的结果及状态更新
合法 VL=0	40	完整旧目的保持及状态策略
非法配置	135	异常与相关状态保持
通过合计	268	上述三类
配置不适用	10	本参考库构建无法通过软件动态关闭 misa.V，单列

API 层使用的本地参考库与服务器参考库身份分别保存。十个不适用用例不计入通过数，原因在于参考库的 RVV 构建配置。每例记录见原始 API 结果〔6〕。

整核阶段再检查译码、执行、异常和退休的联合行为。表 7-2 统一列出定向套件；PIPELINE 小计为三项流水套件之和。

表 7-2　修复版整核差分套件

套件	用例数	目的	结果
SMOKE	7	正负数、极值、掩码、VL=0/16、完整目的与状态	通过
EXCEPTION	64	62 个预期陷阱、2 个合法对照；cause、mepc、mtval	通过
OVERLAP	60	15 种源、目的、掩码相等关系	通过
PIPELINE-DEPS	48	12 个依赖测试程序	通过
PIPELINE-MIXED	32	8 个混发测试程序	通过
PIPELINE-CANCEL-CSR	40	32 个分支场景、8 个精确异常场景	通过
PIPELINE 小计	120	上述三项流水套件	通过

这组回归覆盖表中的固定配置和场景；Hello 与 VADD 也在修复后同轮通过。等价和性能应用另外运行，各自报告调用数，调用数单独统计。原件见修复回归与流水回归〔7〕〔8〕。

7.5 mtval 失配：现象、原因、改法与回归

早期 emu 在短 VL 异常上出现 mtval 失配，根因是执行期非法异常拿不到指令字。RTL 修复后，原异常用例回归，64 例 EXCEPTION 全部通过。下面按现象、原因、改法、回归四步说明。

7.5.1 现象：DUT 的 mtval 为零

早期 emu 执行短 VL 用例 short-1-vm0 时，短 VL 按规格触发非法指令异常，但 DiffTest 报告：

```
mtval different at pc = 0x0080000218, right=0x0020818b, wrong=0
```

参考模型的 mtval 为故障机器码 `0x0020818b`，DUT 的 mtval 为零。结合反汇编和提交轨迹可以区分两个 PC：`0x80000218` 是前一条已提交的 `sd t0,40(s0)`，实际发生异常的 VDot 位于 `0x8000021c`。异常同步时，差分行保留最近的比较位置，`mepc=0x8000021c` 则精确记录了故障指令地址。

```
90    fcsr: 0x0000000000000000 fflags: 0x0000000000000000 frm: 0x0000000000000000
91   ---------------- Privileged CSRs ----------------
92   pc: 0x00000000800002b8  privilege mode: M (mode: 3  v: 0  debug: 0)
93      mstatus: 0x8000040a00007c00   sstatus: 0x8000000200006400  vsstatus: 0x0000000200000000
94      hstatus: 0x0000000200000000  mnstatus: 0x0000000000000008
95       mcause: 0x0000000000000002      mepc: 0x000000008000021c     mtval: 0x000000000020818b
96       scause: 0x0000000000000000      sepc: 0x0000000000000000     stval: 0x0000000000000000
97      vscause: 0x0000000000000000    vsepc: 0x0000000000000000    vstval: 0x0000000000000000
98      mncause: 0x0000000000000000     mnepc: 0x0000000000000000 mnscratch: 0x0000000000000000
99       mtval2: 0x0000000000000000     htval: 0x0000000000000000
00       mtinst: 0x0000000000000000    htinst: 0x0000000000000000
01    mscratch: 0x000000008000bf00  sscratch: 0x9b13456aff208e93 vsscratch: 0x36f2722389759674
02       mtvec: 0x00000000800002b8     stvec: 0x0000000000000000    vstvec: 0x0000000000000000
03         mip: 0x0000000000000000       mie: 0x0000000000000000
04     mideleg: 0x0000000000001444   medeleg: 0x0000000000000000
05     hideleg: 0x0000000000000000   hedeleg: 0x0000000000000000
06        satp: 0x0000000000000000     hgatp: 0x0000000000000000     vsatp: 0x0000000000000000
07    mcounteren: 0x0000000000000000 scounteren: 0x0000000000000000 hcounteren: 0x0000000000000000
08     miselect: 0x0000000000000000  siselect: 0x0000000000000000 vsiselect: 0x0000000000000000
09       mireg: 0x0000000000000000     sireg: 0x0000000000000000    vsireg: 0x0000000000000000
10       mtopi: 0x0000000000000000     stopi: 0x0000000000000000    vstopi: 0x0000000000090001
```

图 7-2　参考快照为 mcause=2、mepc=0x8000021c、mtval=0x0020818b。

```
v28: 0x4538eb0798fcaecc_081e886f2d50a2d0  v29: 0x03ce5ec2bfa5578d_e15cf94a138847a5
v30: 0x7be7ba43cc8318aa_e4a388d363aa0194  v31: 0xcf1c0b672e35e75c_19e7e2e1428a2b0b
   vtype: 0x0000000000000000 vstart: 0x0000000000000000  vxsat: 0x0000000000000001
    vxrm: 0x0000000000000003     vl: 0x0000000000000001   vcsr: 0x0000000000000007
---------------- Triggers ----------------
 tselect: 0x0000000000000000
 0: tdata1: 0xf000000000000000 tdata2: 0x0000000000000000 tdata3: 0x657466666964203a
 1: tdata1: 0xf000000000000000 tdata2: 0x5f64616f6c5f6365 tdata3: 0x722f006d754e6476
 2: tdata1: 0xf000000000000000 tdata2: 0x617461645f657461 tdata3: 0x6e6169782f6a6d2f
 3: tdata1: 0xf000000000000000 tdata2: 0x322d7665642d746f tdata3: 0x2f34323031363230
 4: tdata1: 0x6975622f554d454e tdata2: 0x76637369722f646c tdata3: 0x2d756d656e2d3436
privilegeMode: 3
   mtval different at pc = 0x0080000218, right= 0x000000000020818b, wrong = 0x0000000000000000
Core 0: ESC[31mABORT at pc = 0x8000016e
ESC[0mESC[35mCore-0 instrCnt = 2,742, cycleCnt = 9,606, IPC = 0.285447
ESC[0mESC[34mSeed=1 Guest cycle spent: 9,609 (this will be different from cycleCnt if emu loads a sn
ESC[0mESC[34mHost time spent: 35,252ms
ESC[0m
```

图 7-3　DUT 的 mtval 为零，参考值为故障机器码，差分检查随后中止运行。

7.5.2 原因：执行期非法异常拿不到指令字

短 VL 的非法条件在执行期才产生，而原有 trapInstInfo 通路主要服务于译码期非法指令异常。问题不在八项乘加，而在故障指令走到 ROB 队首、进入精确异常位置时，DUT 没有提供匹配的编码。若只检查目的数值，或关闭这项 CSR 比较，这个集成问题就无法解释清楚。

7.5.3 改法：ROB 分配时保存可变编码字段和代际，队首重建指令字

RTL 在 ROB 分配时保存 VDot 的可变编码字段和代际，队首异常时重建指令字，再由 CSR 选择器优先接收。改法保留原有的异常仲裁和 NEMU 语义，只把缺失的信息接回既有通路。源码与字段容量见第 5 章 5.5。

7.5.4 回归：原异常用例 64 例 EXCEPTION 全部通过

修复生成新 emu 后，我们用原异常镜像与参考库配置回归：64 例 EXCEPTION 全部通过，随后 OVERLAP、SMOKE、Hello 与 VADD 也通过。原失败 emu 和日志保留用于追溯，成功结果对应修复后的仿真器版本。

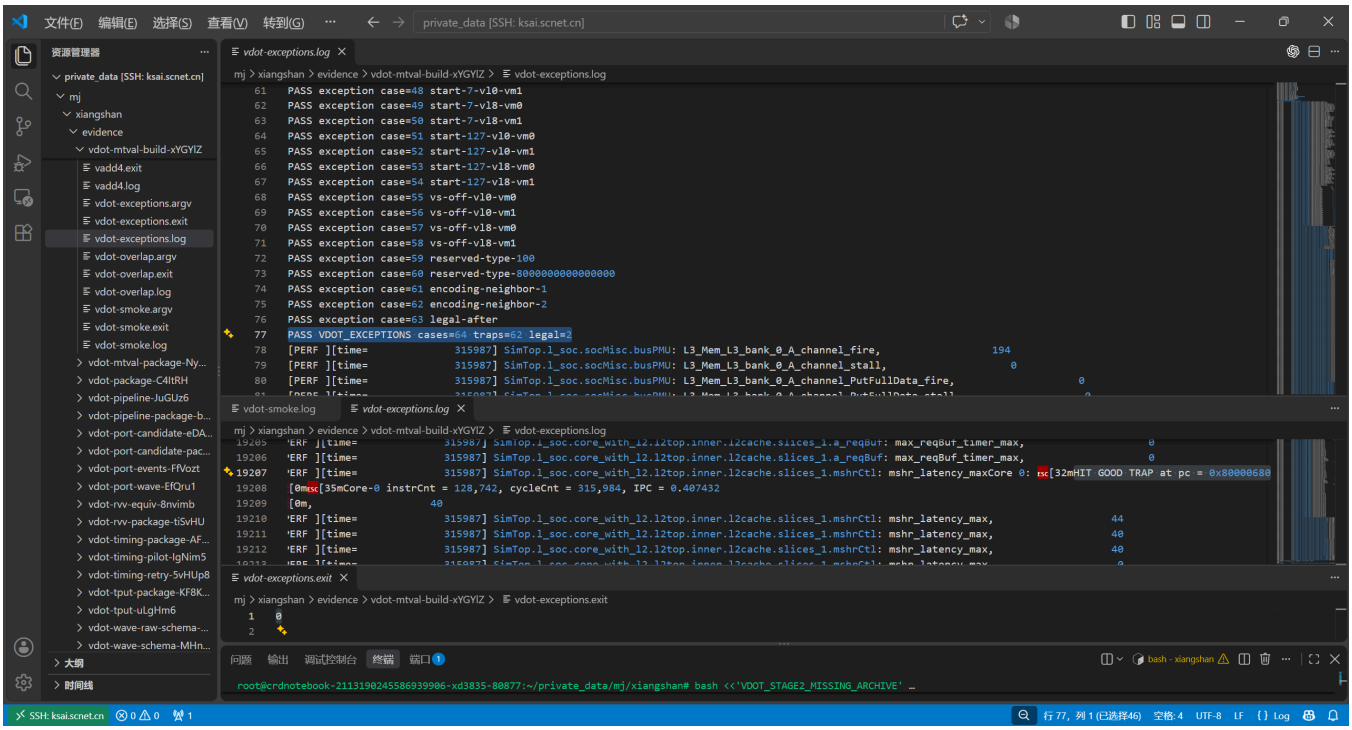


图 7-4　修复后异常套件正常结束，应用 PASS、GOOD TRAP 和退出状态共同确认本次回归结果。

这个案例串起了“发现差异 → 定位故障指令 → 解释缺失信息 → 修改 RTL → 原用例回归”的完整过程。诊断与反汇编依据见差分诊断记录〔9〕，源码修改见修复说明〔10〕。

7.6 OVERLAP 与 PIPELINE 的运行记录

OVERLAP 与三项 PIPELINE 套件的原始运行记录均正常结束，与表 7-2 一一对应。

寄存器重叠检查确认“先读旧源、再写目的”的语义；流水套件检查依赖、混发、分支和精确异常的架构结果。内部动态时序由第 6 章的波形补充。

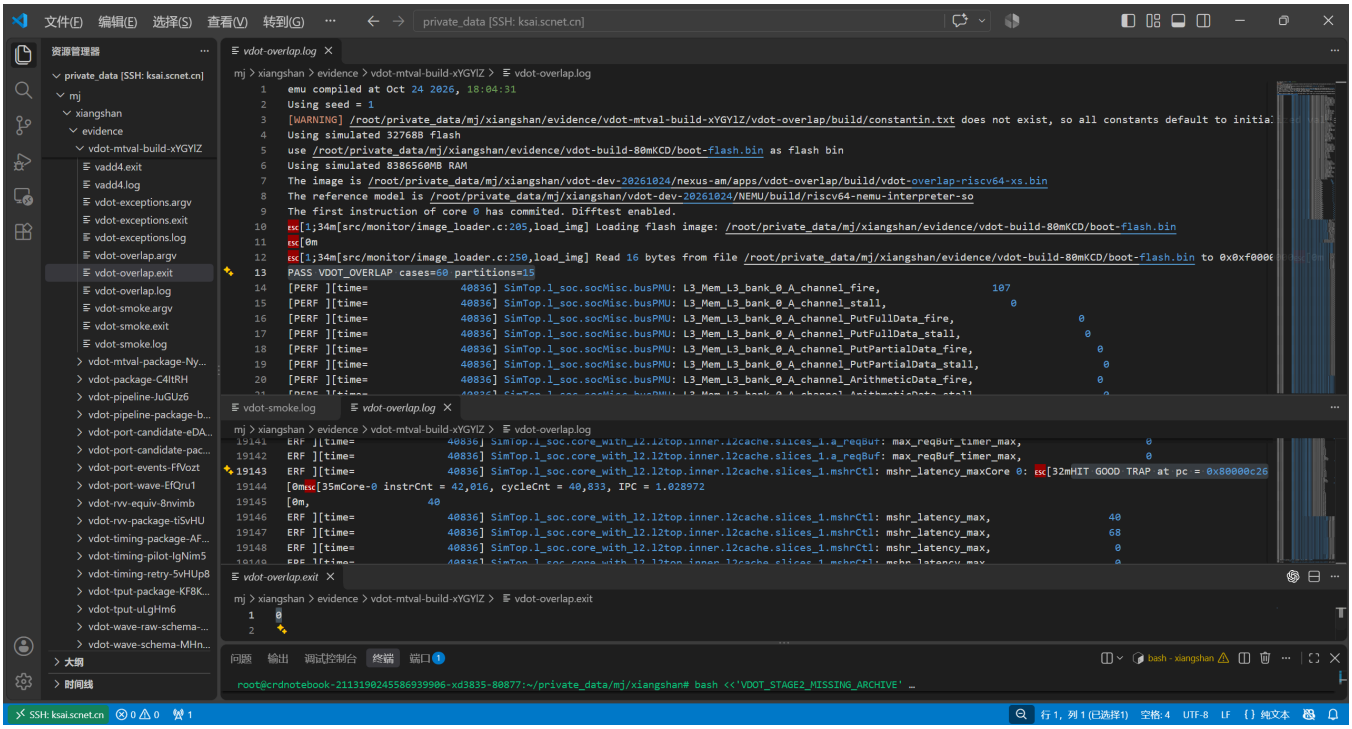


图 7-5　OVERLAP 套件通过，保留 DiffTest、GOOD TRAP 及退出记录。

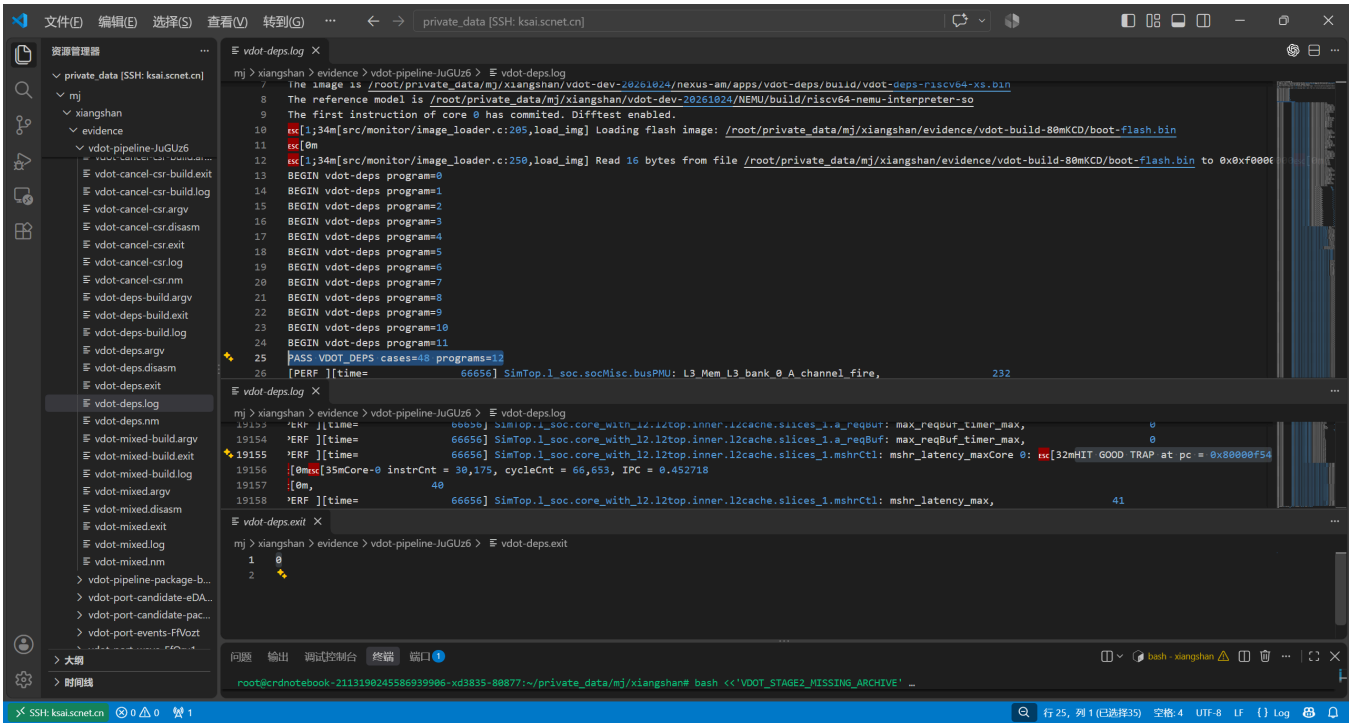


图 7-6　PIPELINE-DEPS 的依赖程序汇总与正常结束记录。

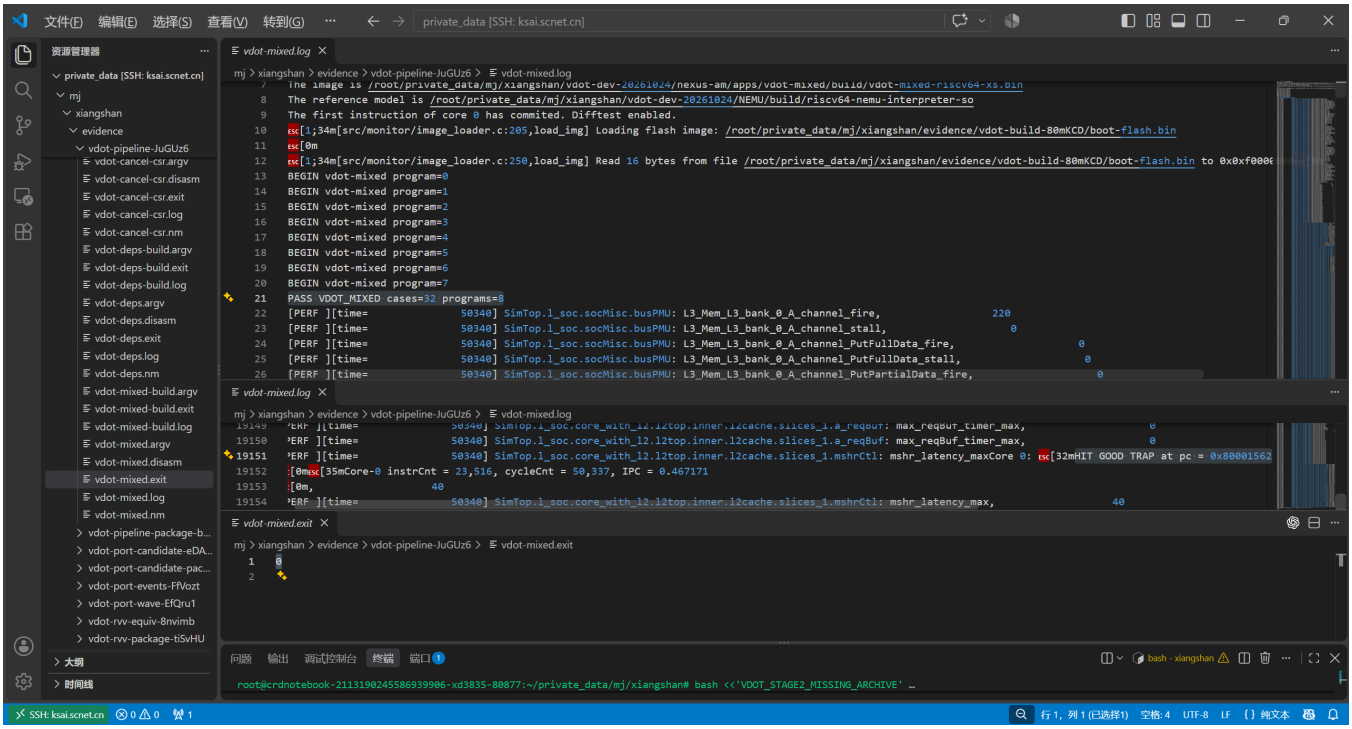


图 7-7　PIPELINE-MIXED 检查选定混发场景的架构结果。

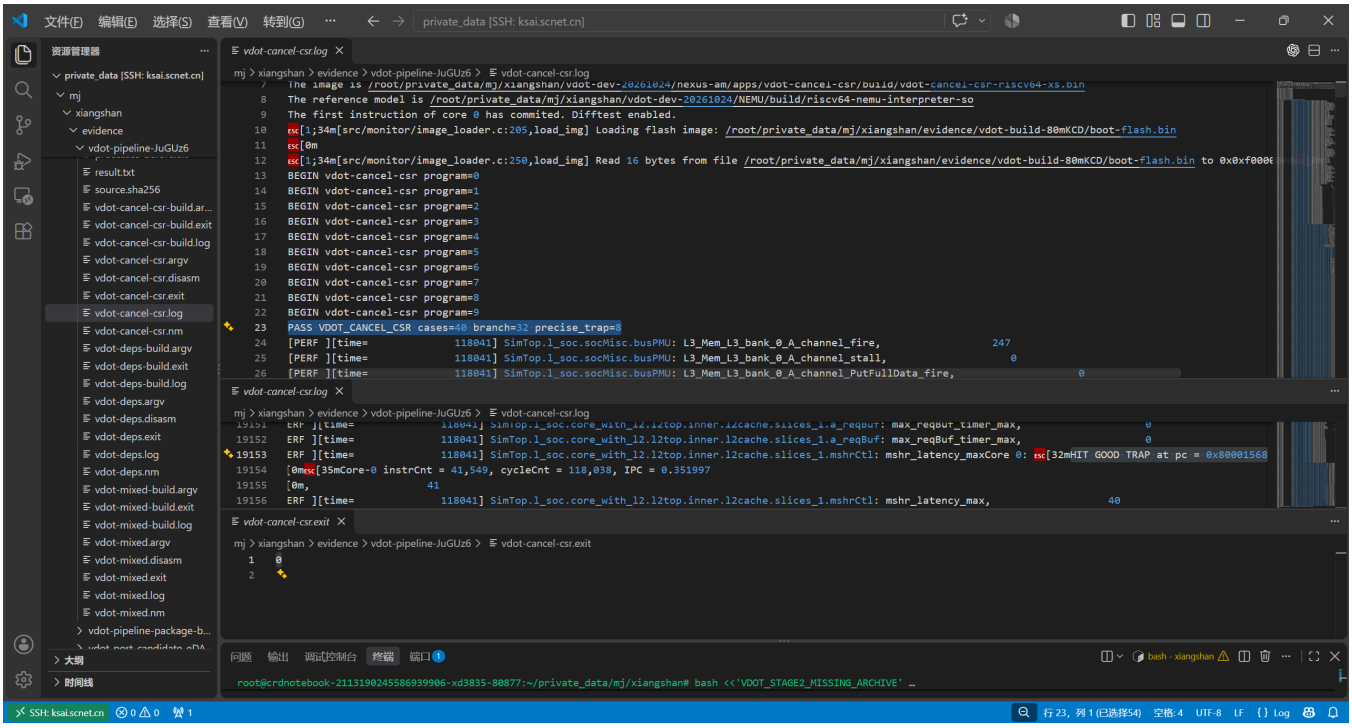


图 7-8　PIPELINE-CANCEL-CSR 分别记录分支和精确异常场景。

在这些记录支持的配置与场景内，参考模型与 RTL 的架构状态保持一致。下一步要建立标准 RVV 组合与自定义指令的等价关系，才能有意义地比较两者的执行成本。

8 标准 RVV 等价基线

两条标准 RVV 基线 RVV-m1、RVV-mf2 与 VDot 在 144 种输入配置、共 432 次内核调用中，结果和指定状态完全一致；本地参考执行、完整 C 检查和服务器 RTL/NEMU 协同运行均通过。

比较延迟之前，先要保证三条路径做的是同一件事。点积的数学和相同还不够：目的高位、VL=0 保持和入口配置恢复同样影响调用者看到的结果。本章用标准 RVV 指令构造两条组合基线，明确各自的寄存器需求，并使用同一组测试用例验证完整结果和指定状态，对应赛题 3.2。

8.1 kernel_end 比较点与寄存器约定

三条路径在汇编 kernel_end 处比较，入口统一为合法 M 态、e8/m1、vstart=0，向量状态可用。

v0 保存掩码，v1/v2 保存源，v3 为目的；v8/v9/v16 和 t0 是允许破坏的临时寄存器。比较位置 kernel_end 位于测试壳再次配置和存储结果之前。

每条路径先与独立期望比较，再做路径间的状态比较。这样既能发现单条实现的错误，也能降低多条实现共享同一错误时漏检的可能。这套调用约定用于性能对比；源与目的寄存器任意重叠、非法配置等情况，由第 7 章的定向套件另行检查。

8.2 vwmul.vv 加 vwredsum.vs：两次加宽得到 INT32

标准 RVV 用 `vwmul.vv` 得到 INT16 乘积，再用 `vwredsum.vs` 加宽归约到 INT32；乘法分组不同，形成 RVV-m1 和 RVV-mf2 两条基线。

`vwmul.vv` 将 INT8×INT8 保存为 INT16 乘积，`vwredsum.vs` 将八项 INT16 加宽归约到 INT32。归约种子必须为零。目的 v3 先完整清零，再借尾部保持策略保留高位零，得到与自定义指令一致的低 32 位结果和高 96 位零。

两次运算需要不同的 SEW/LMUL 配置。有效输入只有八个字节，在 VLEN=128 下有两种合法组织：按 m1 乘法时，加宽目的使用 v8/v9 的 m2 组；按 mf2 乘法时，加宽目的恰好是单个 m1 的 v8。保留两条基线，可以观察寄存器分组怎样影响后续的执行成本。

表 8-1　两条基线的乘法与归约配置

基线	乘法配置	乘积存放	归约配置
RVV-m1	e8/m1，VL=8	EMUL=2，v8/v9	e16/m2，VL=8
RVV-mf2	e8/mf2，VL=8	EMUL=1，v8	e16/m1，VL=8

乘法生成完整的有符号乘积，掩码在归约时选择有效项。VL=0 跳过计算并保留旧目的，末尾恢复调用者的 vl/vtype。标准指令行为依据 RVV 1.0 规格，实际实现以本项目汇编为准〔11〕。

8.3 三条核心段：1 条对 9 条静态指令

完成同样一次点积，VDot 核心段只需 1 条自定义指令，两条 RVV 基线的非零路径各需 9 条静态指令，程序明显更短；实际能快多少，要看第 9 章实测的周期数。

共同测试壳已装入源、掩码和入口配置，a3/a4 分别保存原 VL/vtype。下面列出 vm=0 的三个核心段。

VDot 由单条自定义指令完成：

```
    .word 0x0020818b
```

该字对应 `vdot.vv v3,v2,v1,v0.t`，操作数顺序为 vd、vs2、vs1。

RVV-m1 显式完成分支、配置、乘法、清零、归约和恢复：

```
    beqz a3,rvv_m1_vm0_restore
    vsetivli zero,8,e8,m1,tu,mu
    vwmul.vv v8,v1,v2
    vsetivli zero,4,e32,m1,tu,mu
    vmv.v.i v3,0
    vmv.v.i v16,0
    vsetivli zero,8,e16,m2,tu,mu
    vwredsum.vs v3,v8,v16,v0.t
rvv_m1_vm0_restore:
    vsetvl zero,a3,a4
```

RVV-mf2 改变乘法和归约的分组，其余职责相同：

```
    beqz a3,rvv_mf2_vm0_restore
    vsetivli zero,8,e8,mf2,tu,mu
    vwmul.vv v8,v1,v2
    vsetivli zero,4,e32,m1,tu,mu
    vmv.v.i v3,0
    vmv.v.i v16,0
    vsetivli zero,8,e16,m1,tu,mu
    vwredsum.vs v3,v8,v16,v0.t
rvv_mf2_vm0_restore:
    vsetvl zero,a3,a4
```

完整目的与种子在 e32/m1、VL=4 下清零；切换到归约配置后，tu 保留目的的高位零。末尾恢复原配置，基线不仅返回同一个和，也满足约定的调用状态。vm=1 内核在归约处省去 `v0.t`。

非零路径的九条静态指令包括一条长度分支、三次 vsetivli、一次乘法、两次清零、一次归约和一次恢复。九比一反映本文调用约定下需要写出的指令条数；执行时的依赖等待和配置开销在第 9 章实测。

8.4 等价验证用例：6 组数据、144 种配置、432 次调用

等价验证采用 6 组数据，组合出 144 种输入配置，三条实现共执行 432 次内核调用。C 检查器逐路径比较 v0 ~ v3 共 64 字节，并能拒绝人为扰动的错误结果。

六组数据覆盖正数、混合符号、正负极值、零掩码和稀疏掩码。旧目的与源高半部填入非零值，用来识别错误清零、错误保持和高半输入污染。

表 8-2　测试数据与配置组合

维度	取值
数据集	6 组
vm	0、1
VL	0、8、16
vta/vma	4 种组合
输入配置数	6×2×3×4=144
每配置实现路径	VDot、RVV-m1、RVV-mf2
内核调用总数	144×3=432

Python 整数模型产生独立期望。C 检查器逐路径比较完整的 v0～v3 共 64 字节，并检查约定的 CSR 状态：VL/vtype 恢复，VS=Dirty、vstart=0，vcsr/fflags 保持。源和掩码也在检查范围内，不允许只让 v3 的低位碰巧相同。

我们另对六个内核做了反汇编范围检查，确认标准路径使用加宽乘法与归约，且不含 custom-0 指令。负向检查主动扰动参考模型 RAM 中的期望字节，完整 C 检查器按预期返回 1，说明它能拒绝错误结果。

8.5 整核运行通过，GOOD TRAP 位于 0x80000858

服务器构建、运行、脚本及 tee 退出码均为 0，GOOD TRAP 位于 `0x80000858`；本地参考执行和完整 C 检查也全部通过。

应用独立构建后交给修复版 emu，显式加载源码 NEMU 和共同 Flash。服务器运行使用 seed=1，最大 300 万仿真周期、1800 秒墙钟上限；实际构建 3.726 秒，整核运行 680.930 秒。这些主机时间只用于安排复现，处理器周期另由计时内核读取。

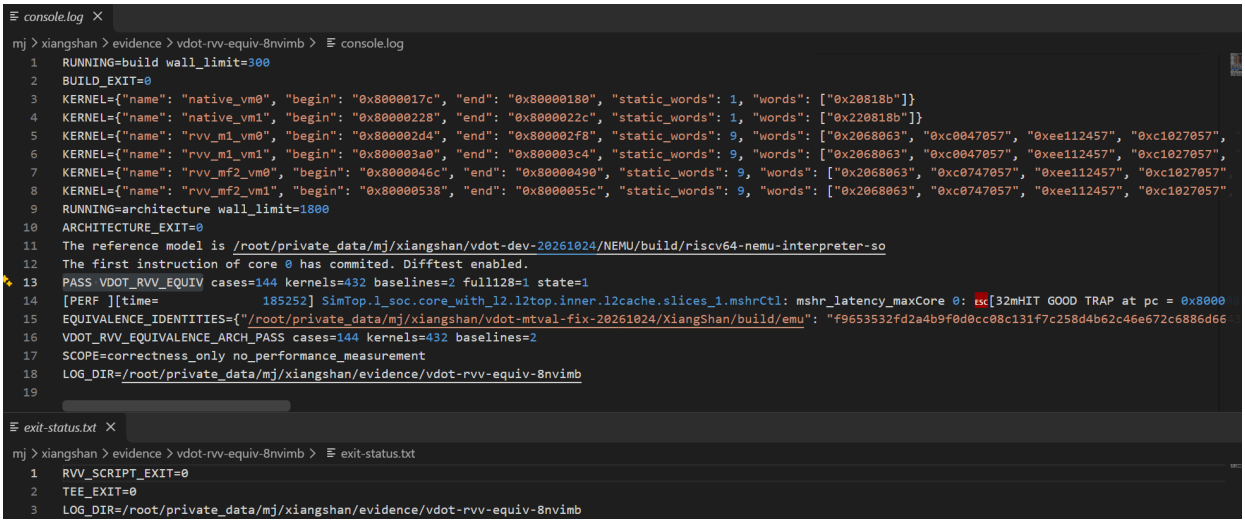


图 8-1　两条 RVV 基线与 VDot 的等价验证通过。正常结束地址在完整日志中为 0x80000858，位于本截图右侧裁剪范围之外。

这组结果建立了固定调用约定下的比较基础。三条路径的结果和指定状态一致；RVV 基线中的配置、清零、归约种子和配置恢复，都是得到与 VDot 相同完整结果的必要步骤。第 9 章把它们纳入各自的计时窗口，先测依赖链延迟，再讨论配置复用和数据搬运改变后的性能。

本地结果、服务器核验、应用源码与实际构建运行入口均保留在配套工程中〔12〕〔13〕〔14〕〔15〕。

9 延迟测量与软件优化分析

在本章固定配置、寄存器输入和完整操作接口下，VDot 的依赖链增量延迟为 7 周期/次，RVV-m1 和 RVV-mf2 分别为 55、47 周期/次，对应延迟比 7.86、6.71。它们包含各实现维持相同接口所需的工作；含访存内核的整窗收益在后续实验中单独报告。

确认三条路径的结果一致后，我们先比较连续相关点积的执行速度：每次点积都使用前一次的结果，输入预先放在寄存器中，测量完整操作的依赖链延迟，对应赛题 3.3。随后测量八路操作的吞吐，再将加载和存储加入计时窗口，观察实际内核中的性能变化。

9.1 用依赖链增量斜率定义延迟

延迟取依赖链中“每多做一次完整操作所增加的周期数”，这样可以消去窗口的固定开销。

9.1.1 从单次耗时转向依赖链增量

直接包围一次操作读取计数器，得到的周期里还包含计时指令本身、读取计数器和使用结果等固定开销。空窗口为 74 周期，单次 VDot 窗口为 81 周期；若只比较这类短窗口的总周期，差异主要由固定开销决定，难以反映指令本身的成本。

主实验展开 N=1、8、32 次点积，每次读写 v1，后一次依赖前一次的结果。输入装载、最终存储和打印放在窗口外；RVV 路径每次仍执行完整的配置、清零和恢复。观察周期随操作数的增长，可以消去各路径的固定截距，得到串行依赖下每次完整操作的增量成本。

9.1.2 周期、延迟、吞吐和加速比

周期取自 RTL 仿真中处理器的 mcycle，退休指令数取自 minstret。服务器耗时是主机执行模拟器的时间，不能换算成被测指令的周期。记 N 次操作的正式样本周期中位数为 C(N)，对应的退休指令增量为 I(N)。

表 9-1　本章的指标定义

指标	定义	含义
窗口周期 C(N)	结束 mcycle − 开始 mcycle，单位为周期	完整计时窗口的执行周期
退休数 I(N)	两次 minstret 读数之差，单位为条	计时窗口内退休的架构指令条数
增量成本 d	$[C(N_2)-C(N_1)]/(N_2-N_1)$，周期/次	每增加一次操作带来的周期增量
完整操作延迟 L	真实依赖链中测得的 d	串行依赖下每次完整操作的周期成本
延迟加速比	L_RVV/L_VDot	RVV 与 VDot 的完整操作延迟之比
增量吞吐比	独立批处理中 d_RVV/d_VDot	操作可重叠时，RVV 与 VDot 的平均周期成本之比
整窗比	同任务、同 N 的 C_RVV/C_VDot	包含公共开销的同任务整窗周期比

加速比大于 1 表示 VDot 的周期成本更低。同一实现的软件优化另用“原组织周期/新组织周期”或耗时减少率 1−C_新/C_原 表达，不与 VDot 和 RVV 之间的加速比混用。周期中位数和最小～最大范围使用全部正式样本；范围反映本轮样本的波动。

9.1.3 预热与取样规则

预热指在正式统计前先调用相同内核，让代码与数据先被访问过，再观察重复调用的成本。预热能减小首次取指、数据访问及其他初始状态的影响，但不保证所有微架构状态都已进入稳态。

同一组的 VDot 和 RVV 变体采用相同的输入准备、预热次数、检查和采样规则，正式轮次轮换执行顺序。预热数据保留，但不进入正式统计。早期实验每变体预热一次，加载与复用补测增加到五次；每点正式样本数见表 9-2。

表 9-2　各实验的样本安排，预热与正式总数包含空对照

实验	变体数	每变体预热	每点正式样本	正式样本总数
计时预检	6	1	6	36
依赖链	10	1	3	30
八路批处理	10	1	3	30
逐点输出小数组	10	1	3	30
分段点积两种组织	13	1	3	39
输入加载策略与轮转	7	5	9	63
2×2 输入复用	5	5	9	45

所有测量采用锁定的单核 MinimalConfig、VLEN=128、seed=1，复用同一修复版 emu、源码 NEMU 和 Flash。早期完整样本见归档逐轮表〔16〕，新增两组见补测结果〔17〕。

9.2 CSRRW 写 mscratch 作计时边界

计时边界采用写 mscratch 的 CSRRW，并在负载后消费结果；预检中每点六次正式样本周期一致，退休增量等于负载指令数加 4。

香山是乱序处理器。仅在负载前后放两条只读 CSR 指令，还需确认它们与负载的执行次序。锁定版本的香山允许部分只读 CSR 指令与前后指令乱序执行，所以我们改用写 mscratch 的 CSRRW 作边界，并在负载后消费运算结果，让结束读数依赖目标工作。

```
外侧边界 → C0 → 边界 → I0 → 边界 → 负载 → 结果消费者
        → 边界 → I1 → 边界 → C1 → 外侧边界 → 保存与恢复
```

C0/C1 读取 mcycle，I0/I1 读取 minstret。预检用 `vmv.x.s` 消费 v3 结果并检查低字节，其他实验按各自的输出设置消费者。mscratch 的入口值保存在 t6，返回前恢复并检查。原始周期差包含四个内部写边界、两次退休计数读取和消费者，我们保留空对照来观察这部分共同成本。

该边界针对本版本微架构的调度行为建立，实际汇编见计时程序〔18〕。含存储的实验还在窗口末尾执行一次 `fence rw,rw` 和结果读回，把最终输出完成的成本纳入比较。

预检用空操作、整数指令序列和三条点积路径检查边界，结果见表 9-3。

表 9-3　计时预检结果

负载	负载指令数	窗口周期	退休增量
empty	0	74	4
addi16	16	86	20
addi64	64	134	68
VDot	1	81	5
RVV-m1	9	130	13
RVV-mf2	9	122	13

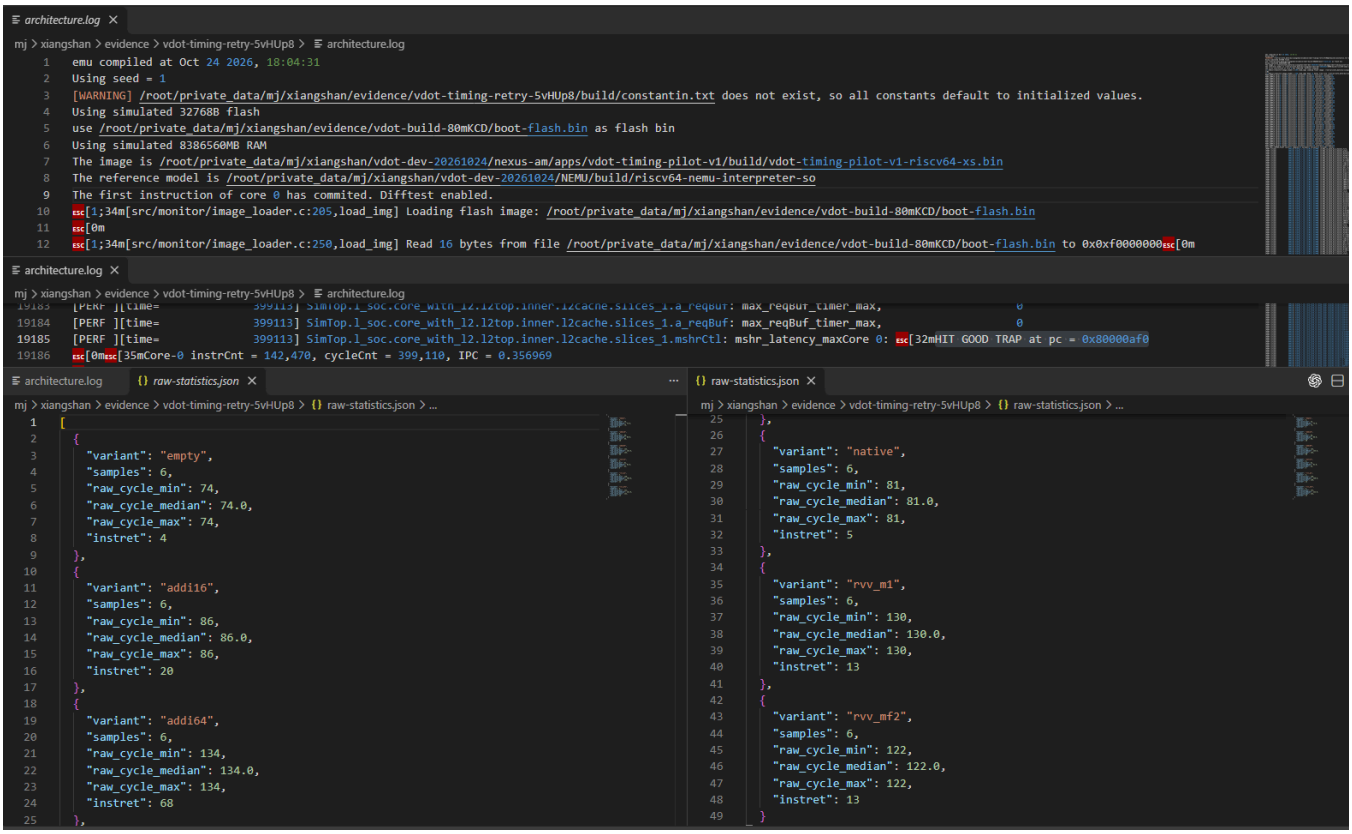


图 9-1　六组预检的周期与退休数，连同参考库、DiffTest 和正常结束信息。应用通过及退出状态保存在预检记录〔19〕。

9.3 依赖链延迟：7 对 55 / 47 周期

依赖链中 VDot 为 7 周期/次，RVV-m1、RVV-mf2 为 55、47 周期/次，延迟加速比为 7.86、6.71。

依赖链的每次点积都读写 v1，后一次必须等前一次的结果。窗口内包含 RVV 每次的配置、乘法、清零、归约和恢复，VDot 则用自定义指令完成规定操作。测量的是相同接口下完整操作的推进速度。表 9-4 列出三种长度的原始中位数；各点三个正式样本一致，两段长度扫描的斜率也相同。

表 9-4　依赖链周期和退休数，单元格为“周期 / 条”

路径	N=1	N=8	N=32	1→8 增量	8→32 增量
VDot	81 / 5	130 / 12	298 / 36	7 周期/次	7 周期/次
RVV-m1	130 / 13	515 / 76	1835 / 292	55 周期/次	55 周期/次
RVV-mf2	122 / 13	451 / 76	1579 / 292	47 周期/次	47 周期/次

7 周期是完整操作在后端依赖、调度和执行路径上的增量成本；第 6 章波形中的 2 周期对应 FU 从输入到完成，前者覆盖整条后端路径，后者只计功能单元内部，两者测量范围不同，可互为补充。

以 8→32 为例，VDot 的增量为 (298−130)/24=7，RVV-mf2 为 (1579−451)/24=47。由此：

```
相对 RVV-m1: 55 / 7 = 7.86 倍
相对 RVV-mf2: 47 / 7 = 6.71 倍
```

三个点分别满足 C_VDot(N)=74+7N、C_m1(N)=75+55N、C_mf2(N)=75+47N。RVV 的截距比空对照多 1 周期。用相邻点差分可以消去各自的固定截距，不必假定三条路径共享同一个截距。

N=32 的 RVV-mf2/VDot 整窗比为 1579/298=5.30，小于增量比 6.71，原因是窗口仍包含固定开销。

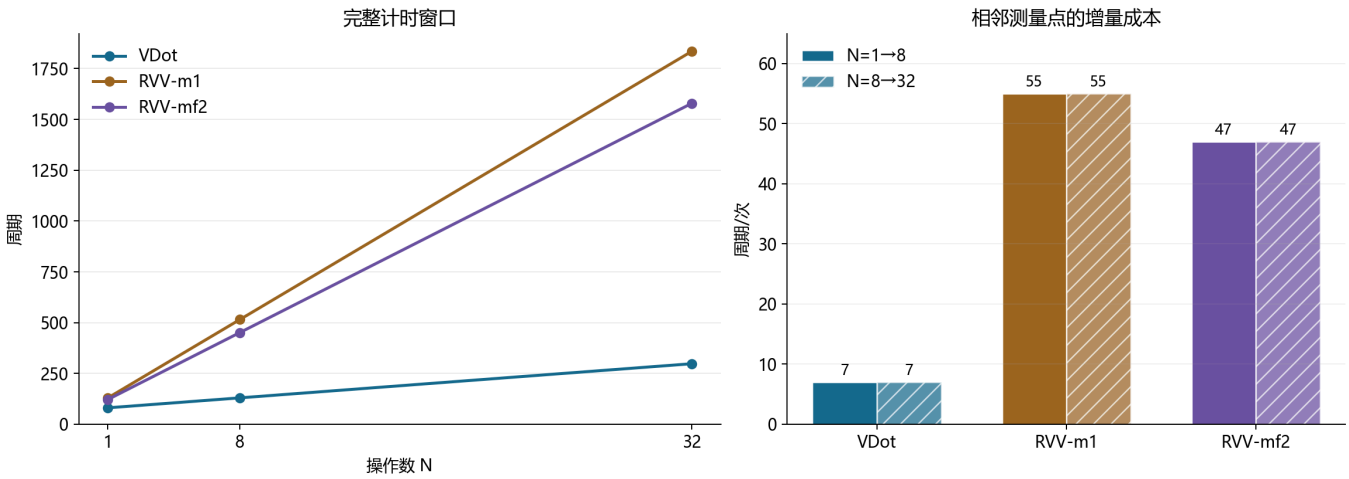


图 9-2　依赖链数据与斜率。VDot 两段均为 7 周期/次，两条 RVV 基线分别为 55、47 周期/次。

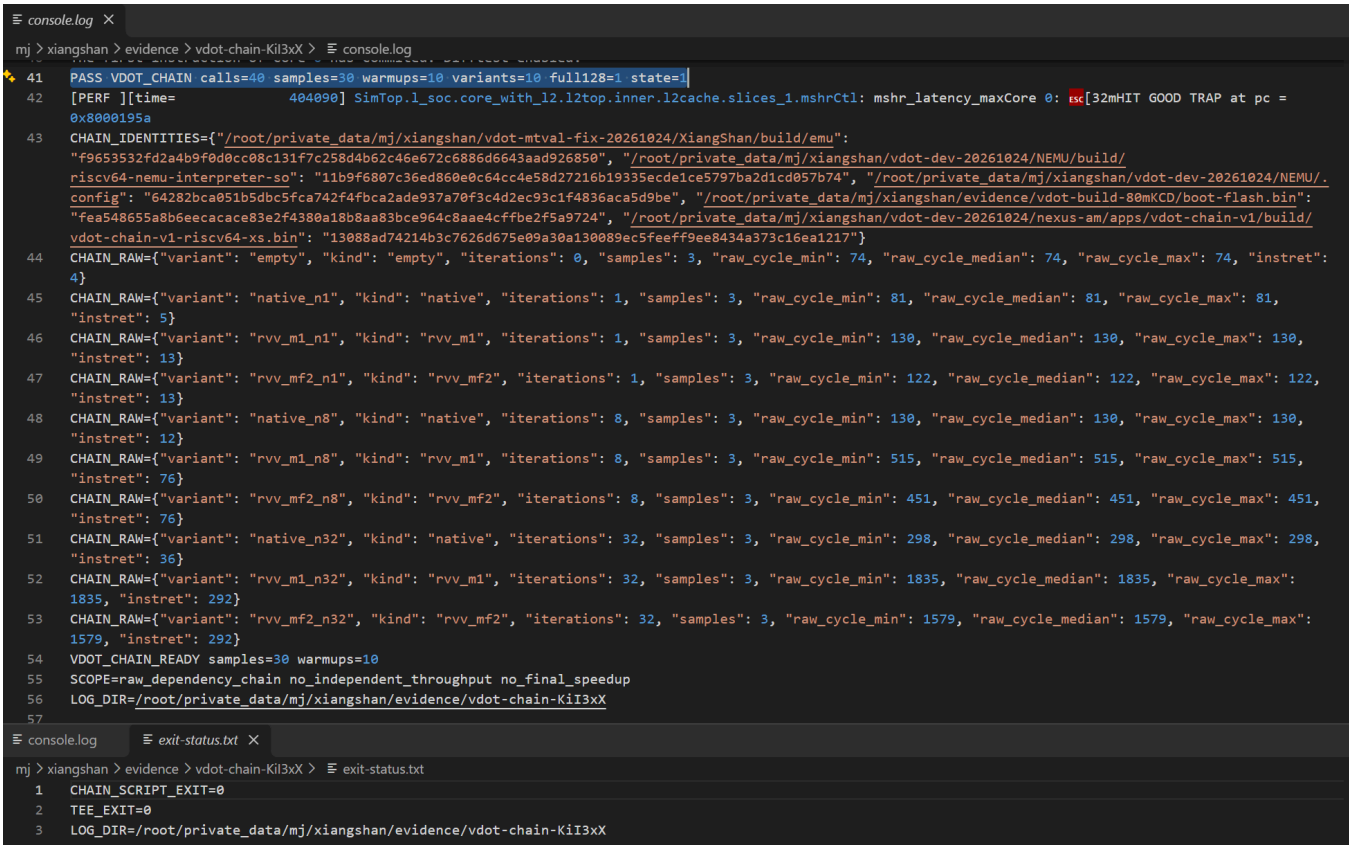


图 9-3　依赖链原始记录，包含周期、退休数、PASS 和退出状态。数据来源为原始统计〔20〕。

这组依赖链数据用于赛题要求的 Latency 对比。在完整操作接口和固定配置下，VDot 连续执行相关点积所需的周期明显更少。源码、运行和样本见依赖链程序及运行记录〔21〕〔22〕。

9.4 八路批处理：VDot 增量 1 周期/次

减少结果回授并共享配置后，VDot 的增量成本为 1 周期/次，相对 RVV-m1、RVV-mf2 的增量吞吐比为 6 和 3。

实际计算不一定每次都等待上一结果。第二组实验固定 v1/v2 为输入，把结果轮转保存到 v24～v31，减少结果到输入的回授；RVV 每八次共享乘法和归约配置。窗口包含起始清零、结束恢复和八个结果的读取汇总；目的复用仍保留既有的 oldVd 依赖。

表 9-5　八路批处理周期和退休数，单元格为“周期 / 条”

路径	N=8	N=32	N=128	8→32 增量	32→128 增量
VDot	96 / 26	120 / 50	216 / 146	1.00	1.00
RVV-m1	186 / 47	330 / 101	906 / 317	6.00	6.00
RVV-mf2	159 / 47	232 / 101	519 / 317	3.04	2.99

增量单位为周期/次，各点三个正式周期样本一致。VDot 满足 C(N)=88+N，RVV-m1 为 138+6N。RVV-mf2 在 8→128 的平均斜率为 (519−159)/120=3，中间测量点比两端连线高 1 周期。

N=128 的整窗比为 906/216=4.19、519/216=2.40。这些数字描述多次操作重叠执行后的平均成本，属于吞吐指标，不再计入单条依赖延迟的加速结论。

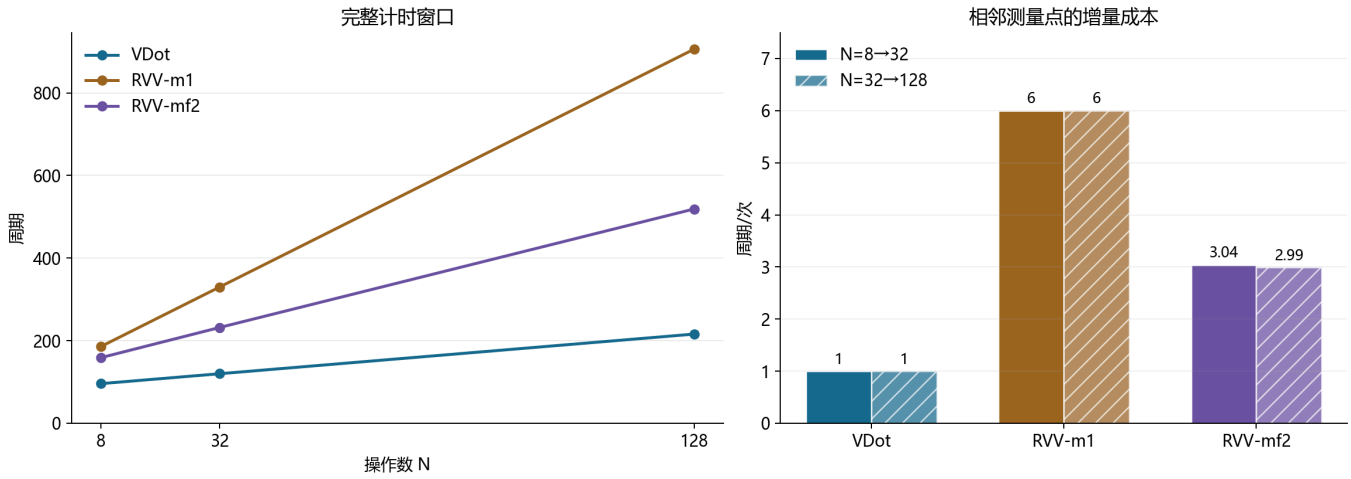


图 9-4　VDot 的增量为 1 周期/次；RVV-mf2 保留两段实际差分值。

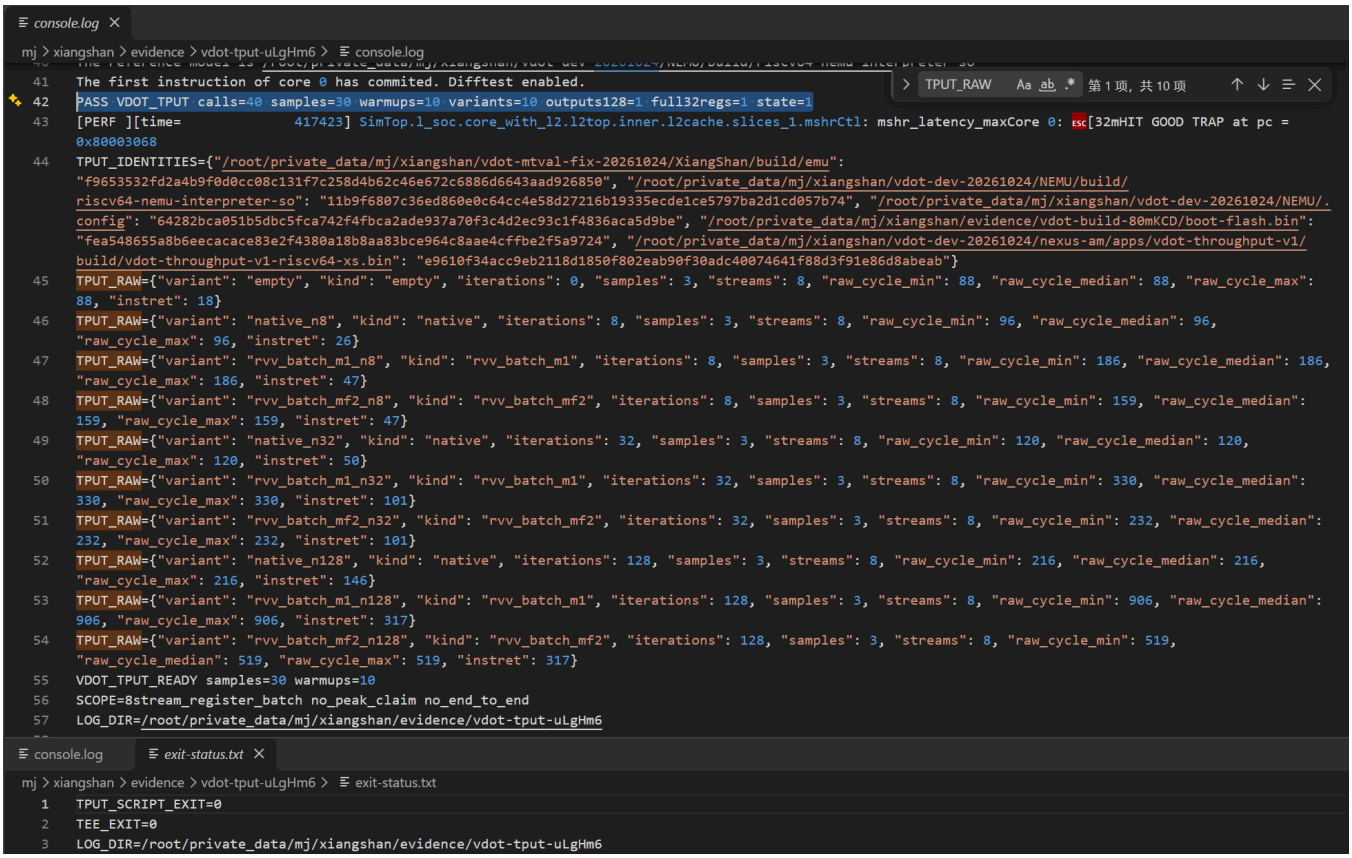


图 9-5　八路批处理实测结果，对应原始统计〔23〕。

寄存器内的延迟和吞吐优势已经确立。下一步把输入从内存送入、把结果写回，看这些优势在更完整的软件窗口里还剩多少。

9.5 逐点输出小数组：N=128 时基本持平

加入加载、存储和排空后，N=128 时 VDot 与 RVV-mf2 基本持平，整窗比为 2241/2239=1.0009，算术优势被数据搬运稀释。

小数组内核每次点积读取两组各八字节输入，并保存完整的 16 字节输出。程序每算完八项就集中存储八个结果，在整个窗口末尾执行一次 fence 和末项读回；初始化、输出哨兵、自检和打印位于窗口外。窗口内同时包含地址计算、加载、配置、点积、存储和排空。

表 9-6　逐点输出小数组的实测成本。周期为中位数，括号为出现波动时的最小～最大值；每点三个正式样本

路径	N	周期	退休数
VDot	8	287	65
VDot	32	674（674～676）	239
VDot	128	2239（2239～2240）	935
RVV-m1	8	320	84
RVV-m1	32	705	285
RVV-m1	128	2249	1089
RVV-mf2	8	312	84
RVV-mf2	32	698（696～698）	285
RVV-mf2	128	2241	1089

N=128 时，VDot 比 RVV-mf2 少退休 154 条指令，却只少 2 周期。VDot 的整窗平均成本约为 17.49 周期/次，明显高于寄存器批处理的增量成本。缩短算术序列并没有同比例缩短这个含数据搬运的窗口。

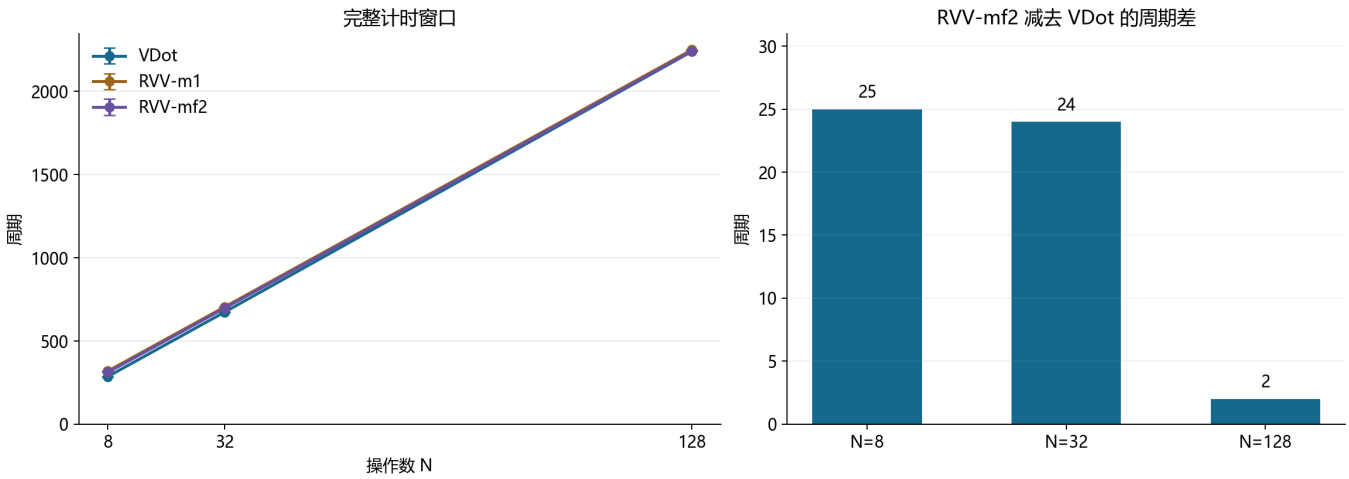


图 9-6　随着 N 增长，VDot 与 RVV-mf2 的周期差从 25、24 缩小到 2。

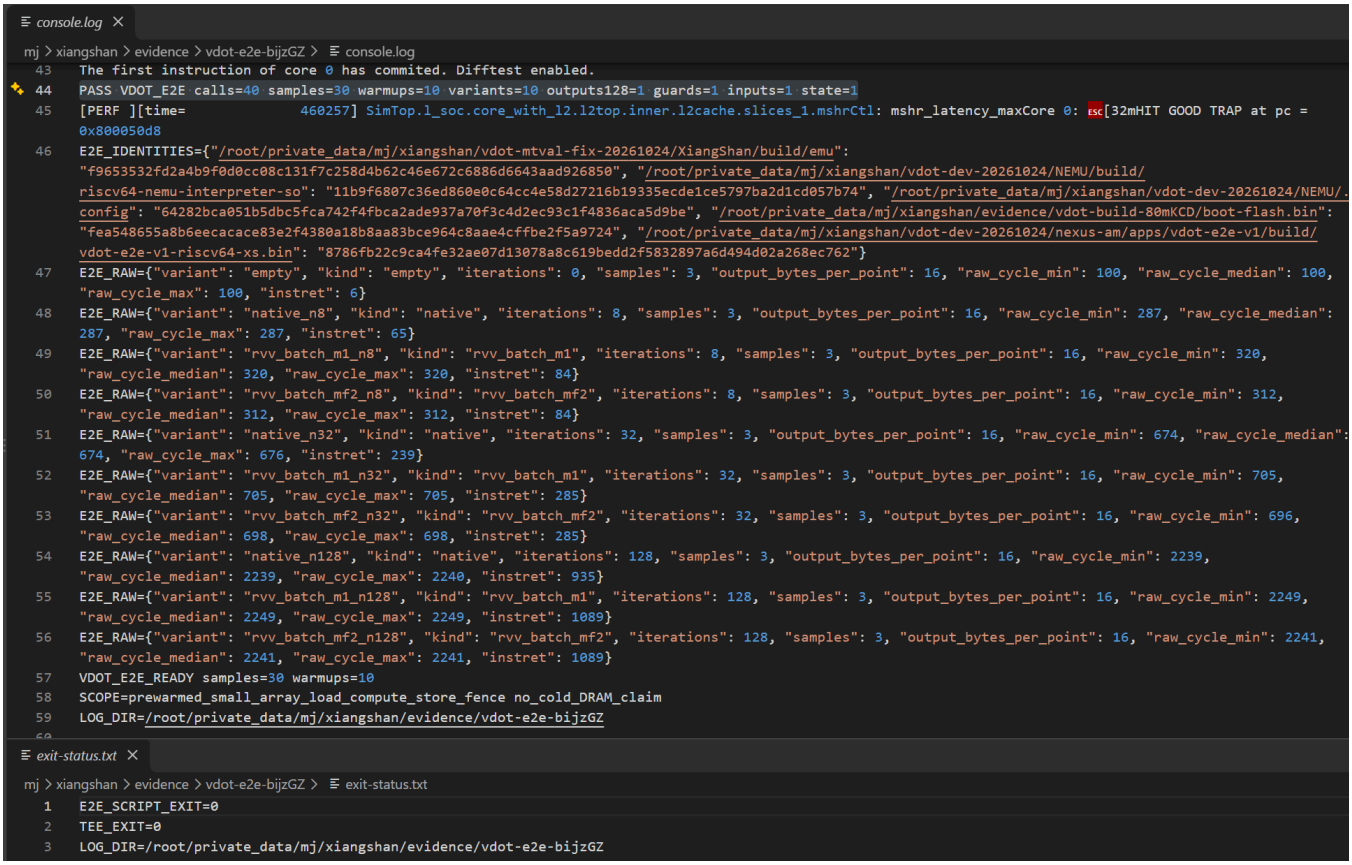


图 9-7　小数组原始统计及检查记录；完整数据见原始统计〔24〕。

全应用计数器指出了进一步检查的方向：加载和存储等待较多，Sbuffer 请求的有效次数与接受次数差异明显。

表 9-7　小数组应用全程计数，日志 time=460257

计数器	值	观察含义
ROB clock_cycle	460258	整个应用的计数范围
ROB commitInstr	210343	整个应用退休量
ROB waitLoadCycle	182322	存在较多加载等待活动
ROB waitStoreCycle	161379	存在较多存储等待活动
ROB waitFenCycle	860	包含 fence 等待
Sbuffer req_valid / req_fire	31229 / 16772	请求与接受次数不同
DCache req_valid / req_fire	3910 / 3903	该接口请求和接受计数

计数覆盖预热、初始化、自检、打印和所有变体，各等待项也可能重叠。这些数据提示应继续检查数据搬运与排空；由于无法从中拆出单个内核的停顿比例，也不足以断定哪一种资源是主要瓶颈。原始字段见完整计数器路径〔25〕。

据此我们设计了三个分别受控的同输出对照：对照A 在相同段和输出下减少中间搬运；对照B 固定分段任务，调整输入加载；对照C 在共享输入的 2×2 任务中减少加载次数。

9.6 对照A：分段归约省去中间落存

结果：省去中间落存后，VDot 在 G=8、G=16 中耗时分别减少 24.7%、26.0%；同为寄存器归约时，VDot 相对 RVV-mf2 的整窗比仅为 1.017、1.042。

新任务每窗口读取两组各 1024 字节输入，完成 128 次八项点积，每连续 G=8 或 G=16 次得到一个段和。两种组织产生相同的段和：一种直接在寄存器中归约；另一种先把所有点积结果存入 scratch，再读回归约。后者额外搬运 2048 字节写和 2048 字节读。

G=8 最终输出 16 个完整向量结果，G=16 输出八个。两种组织、三条实现各自比较；计时均包含加载、地址与循环控制、计算、归约、最终存储及末尾 fence/读回。

表 9-8　分段任务的相同输出对照。括号为正式周期范围，退休数按“寄存器 / 落存”列出

G	路径	寄存器归约周期	中间落存周期	退休数
8	VDot	2282（2282～2383）	3031（3030～3169）	859 / 1388
8	RVV-m1	2275（2273～2427）	3394（3394～3440）	1013 / 1542
8	RVV-mf2	2321（2320～2454）	3312（3312～3313）	1013 / 1542
16	VDot	2240（2240～2296）	3026（3026～3079）	843 / 1372
16	RVV-m1	2360（2359～2438）	3443（3443～3444）	997 / 1526
16	RVV-mf2	2333（2333～2405）	3263（3262～3312）	997 / 1526

两条 RVV 路径省去落存后也有改善，说明减少中间搬运对三种实现都有效。因此，调整数据组织带来的收益应与 VDot 指令本身带来的收益分开计算。

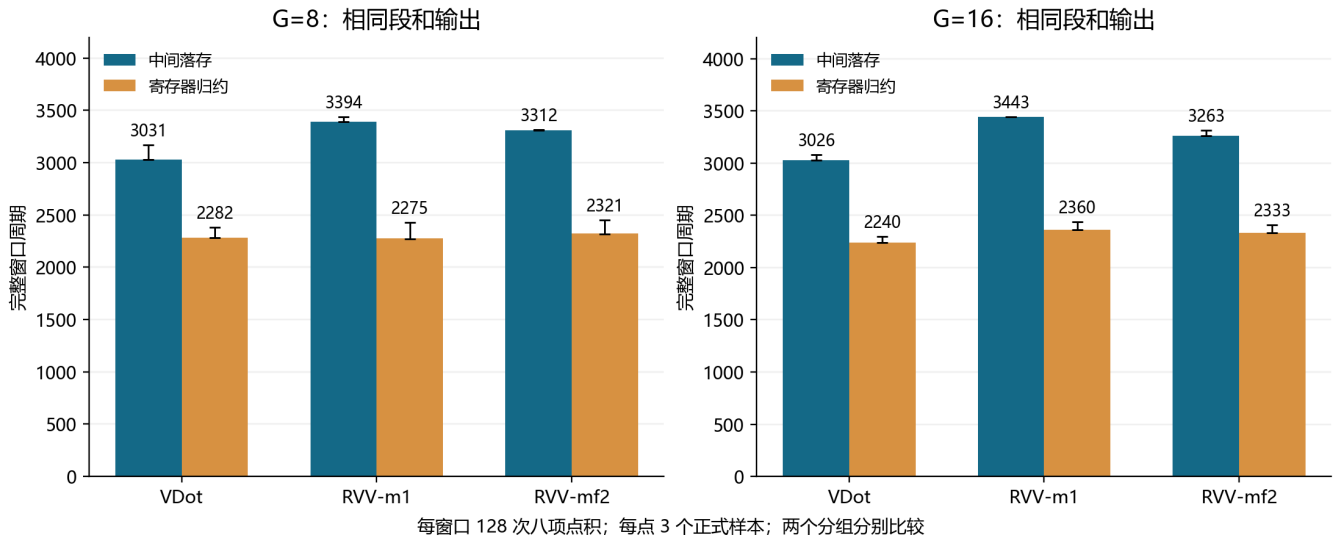


图 9-8　G=8、G=16 分面展示，各面只比较相同段和输出。柱高为中位数，误差线为三个正式样本范围。

多数变体的首个正式样本仍偏高。例如 G=8 的 VDot 为 2383、2282、2282 周期，RVV-m1 为 2427、2273、2275。小幅的跨实现差异需要结合这一波动来读。原始样本与统计见分段核验〔26〕。

这项实验只需输出段和，因此可以在寄存器中直接归约，省去中间结果的写入和读回；9.5 则需要逐点保存结果。减少中间搬运后，每次点积仍需加载输入，接下来继续调整加载策略和指令安排。

9.7 对照B：调整输入策略和加载安排

结果：加载条数同为 256，ta_rr 组织使 VDot 从 2239 降到 1089 周期，耗时减少 51.4%；同组织下 VDot 相对 RVV-mf2 的整窗比为 1.24。

为检查输入端成本，我们固定 G=16 任务：每窗 128 次点积，最终输出八个段和，执行 256 条向量加载。共设三个组织：原 tu；仅改变输入尾部策略的 ta；在 ta 基础上用 v1/v2、v6/v7 两组寄存器轮转并提前加载的 ta_rr。两条实现接受相同的优化，VDot 自身的 oldVd 语义不变。

这组实验重新测量了 tu 对照，采用每变体五次预热、九次正式样本。历史 G=16 的 2240/2333 周期来自不同的程序上下文和较少的预热，本节的优化比全部使用本批数据。

表 9-9　同一 G=16 任务的加载对照，每种组织均为 256 条向量加载

组织	路径	周期中位数	最小～最大	退休数
tu	VDot	2239	2238～2241	843
tu	RVV-mf2	2262	2260～2262	997
ta	VDot	1235	1231～1236	843
ta	RVV-mf2	1396	1392～1398	997
ta_rr	VDot	1089	1087～1090	843
ta_rr	RVV-mf2	1355	1352～1357	997

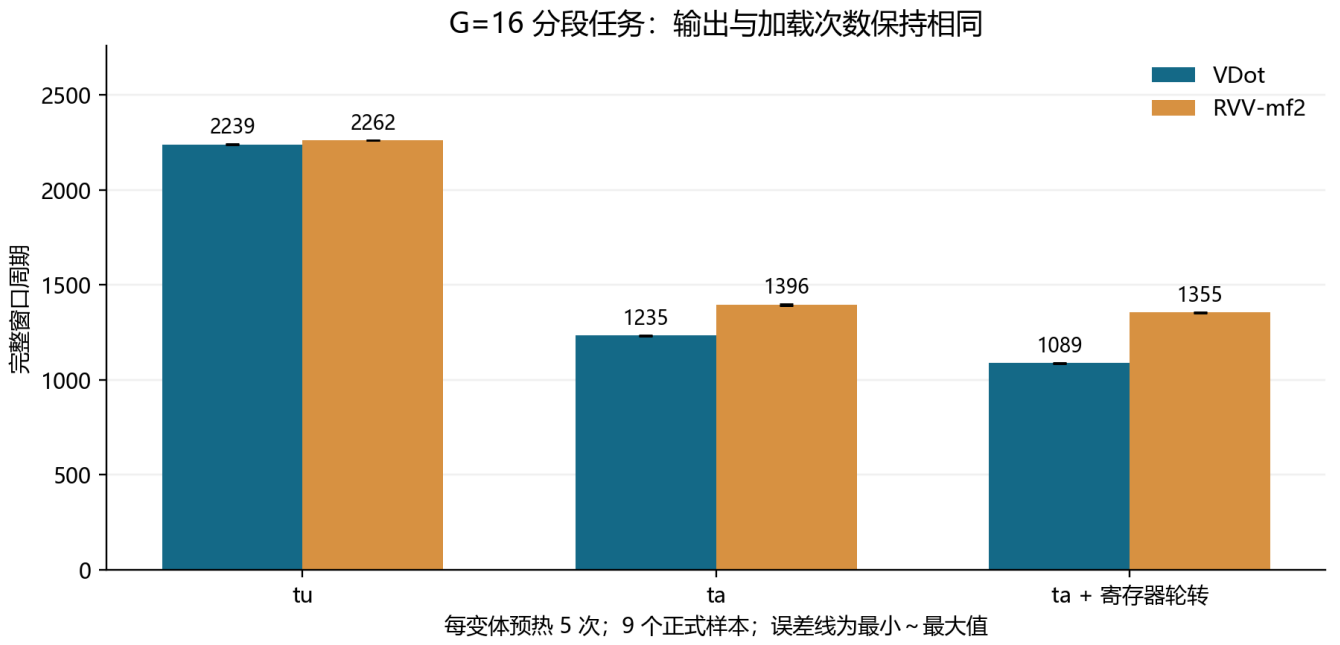


图 9-9　同输出、同加载条数下，策略和安排改变后周期下降。误差线为九个正式样本范围。

RVV-mf2 从 2262 降到 1355 周期。两条路径各自的退休数保持不变。周期下降与减少旧值保持约束、调整加载顺序的设计动机一致：ta 不再要求保留输入寄存器尾部的旧值，寄存器轮转则允许提前安排下一组加载。现有数据确认这组软件变化有效，但尚未通过逐内核事件拆分依赖等待、调度和其他因素各自的贡献。

在同样的 ta_rr 组织下，VDot 相对 RVV-mf2 的整窗比为 1355/1089=1.24，耗时减少 19.6%。原始轮次、运行身份和检查见加载实验记录〔27〕。

9.8 对照C：2×2 小块复用输入

结果：复用输入把向量加载从 256 条减到 128 条，VDot 从 1077 降到 790 周期，减少 26.6%；同组织下 VDot 相对 RVV-mf2 的整窗比为 1.32。

对照B 没有减少加载条数。本组考察共享输入的多输出任务：计算 K=256 的 2×2 小块，四个输出分别累加 A0/B0、A0/B1、A1/B0、A1/B1 的点积。每轮处理八个元素，共 32 轮，即每窗 128 次八项点积，最终保存四个 128 位结果。

独立加载组织为每个点积重新装入两个输入，每轮八次加载。复用组织只加载 A0、A1、B0、B1 四个块，再计算四个点积，完整窗口的向量加载从 256 条减到 128 条。双方采用相同的 ta 策略和预先按列存放的 B 输入；布局准备位于计时窗口外。

表 9-10　同一四输出任务的输入复用比较，五次预热后取九个正式样本

组织	路径	周期中位数	最小～最大	退休数	向量加载数
独立加载	VDot	1077	1076～1087	950	256
独立加载	RVV-mf2	1268	1262～1270	1115	256
复用加载	VDot	790	787～792	694	128
复用加载	RVV-mf2	1043	1041～1044	859	128

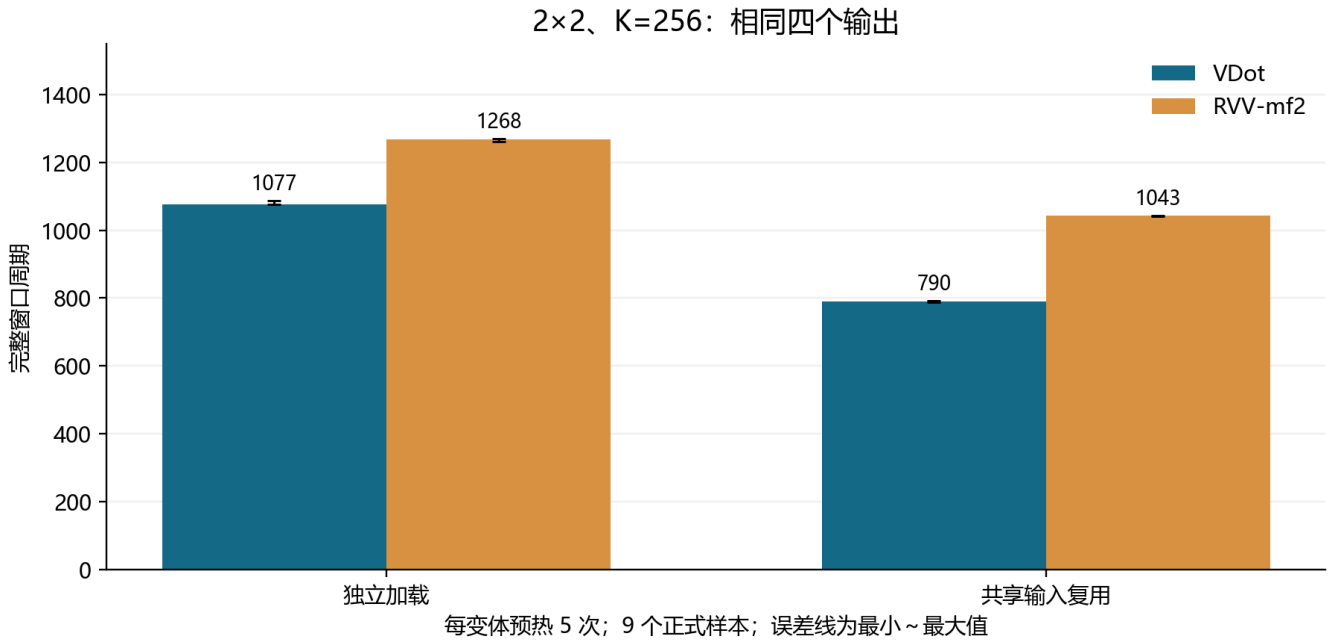


图 9-10　相同四个输出下的复用收益。该图与分段任务分别比较，不连接为跨任务趋势线。

RVV-mf2 从 1268 降到 1043 周期，减少 17.7%。两条路径各少退休 256 条指令，包含向量加载及地址推进的减少。复用组织内，VDot 相对同样优化的 RVV-mf2 为 1043/790=1.32 倍整窗比，耗时减少 24.3%。

这组结果体现了共享输入的作用。适用范围为固定配置、固定种子和预热后的给定 2×2 内核，不包含矩阵转置或打包成本。完整样本见复用实验记录〔28〕；完整输出、保护区、输入、指定状态及差分检查的核验见两组汇总〔17〕。

9.9 性能结论与适用条件

- **延迟**：完整操作依赖链中，VDot 为 7 周期/次，RVV-m1、RVV-mf2 为 55、47 周期/次，延迟加速比 7.86、6.71。八路批处理中 VDot 增量成本为 1 周期/次，增量吞吐比为 6、3；吞吐数字不作为延迟重复计分。
- **适用条件**：优势在寄存器内的依赖链和共享配置的批处理中最明显。加入访存后，逐点完整输出内核在 N=128 时基本持平；在另行定义的相同输出任务内，减少中间存储、调整输入策略与加载安排、复用共享输入都能降低成本，同组织跨实现比较分别为 1.24（G=16 加载轮转）和 1.32（2×2 复用）。应用侧既要缩短点积核心序列，也要避免不必要的中间存储，并结合有效输入范围安排加载与复用。
- **不主张的内容**：结果限于单核 MinimalConfig、VLEN=128、seed=1 和预热后的给定内核。完整应用、冷数据及其他处理器配置的收益需要相应实验，不能由某一个内核比值直接推定；2×2 结果不含矩阵转置或打包成本。

附录 C 模型与性能索引

本附录列出第三阶段的固定环境、应用与复现入口。第三阶段复用第二阶段的修复版 emu、源码 NEMU 和共同 Flash，性能运行固定 seed=1。本文中的历史核验与本次文档核对分开记录；本次未启动新的整核构建或仿真。

C.1 固定环境与模型

表 C-1　第三阶段主要身份

对象	版本或保存的 SHA256
XiangShan 基版本	`16ae9ddcda54fc9a2fddffad73174cf793ac7814`，叠加完整 RTL 补丁
NEMU 基版本	`a48a11f208ff81ad765591d83271d1b339fb3861`，叠加 VDot 参考语义
emu	`f9653532fd2a4b9f0d0cc08c131f7c258d4b62c46e672c6886d6643aad926850`
服务器参考库	`11b9f6807c36ed860e0c64cc4e58d27216b19335ecde1ce5797ba2d1cd057b74`
Flash	`fea548655a8b6eecacace83e2f4380a18b8aa83bce964c8aae4cffbe2f5a9724`
本地 API/等价检查参考库	`5ba6caf77513904c6ea541c4c145720a6bebe63f983eefa5b73e53441069c872`

本地 API 和服务器运行分别采用上述参考库身份。具体依赖和 NEMU 配置见源码交付记录〔5〕；每个应用的运行参数、二进制身份和退出状态位于各自的原始记录中。

C.2 应用、样本和复现入口

各应用均独立生成 RISC-V 镜像，由固定 emu 加载，并显式设置 `--diff` 与 `-F`。原运行入口保存了输入包、构建参数和日志目录的创建方式。在新环境独立复验时，应使用新的运行目录并核对参考库身份。

表 C-2　应用与对应证据

实验	源码	运行与数据
RVV 等价	vdot-rvv-equiv-v1〔14〕	命令〔15〕、结果〔13〕
计时预检	vdot-timing-pilot-v1〔29〕	运行入口〔30〕、统计〔31〕
依赖链	vdot-chain-v1〔32〕	命令〔33〕、统计〔20〕
八路批处理	vdot-throughput-v1〔34〕	命令〔35〕、统计〔23〕
小数组	vdot-e2e-v1〔36〕	命令〔37〕、统计〔24〕
分段点积	vdot-segment-v1〔38〕	命令〔39〕、完整样本〔26〕
输入加载与轮转	方法及源码入口〔40〕	全部轮次〔27〕
2×2 输入复用	方法及源码入口〔40〕	全部轮次〔28〕

两组加载/复用应用共 168 次调用，其中预热 60 次、正式 108 次，来源为汇总核验〔17〕。加载恢复运行复用了已构建的镜像，首次中断的目录和原构建耗时未完整归档；成功运行的源码、BIN、ELF、样本和身份均已保存。

C.3 应用镜像与完整原件

表 C-3　主要应用 BIN 的保存 SHA256

应用	SHA256
RVV 等价	bf29b9367a4b9045b139224bb5b2b6502bdeae005969abdc6129ba0fd6c76010
计时预检	8545abf1afb71ec405a8c81bada3fe5054a332964436eb7976a824edaf91bda6
依赖链	13088ad74214b3c7626d675e09a30a130089ec5feeff9ee8434a373c16ea1217
八路批处理	e9610f34acc9eb2118d1850f802eab90f30adc40074641f88d3f91e86d8abeab
小数组	8786fb22c9ca4fe32ae07d13078a8c619bedd2f5832897a6d494d02a268ec762
分段点积	bd034d3d3bebae84c68d26d5b2d52c6420f26ea89c41178621a217321c0aac8b
输入加载	6436b127636a046afa8e9451270be3a48023ff21f4b86a0c1c194e41d217f54f

模型及故障定位原件包括 NEMU 源码快照、API 结果、差分失败诊断、修复回归及流水套件〔3〕〔6〕〔9〕〔7〕〔8〕。

早期等价及四组计时的完整来源文件保存在本地原件归档中〔41〕，其完整逐轮样本支持复算〔16〕。后续分段和加载/复用各有独立的原始目录。复核已有结论时优先从这些原件读取；需要新实验时，再明确配置、输入、样本和计时边界。

材料完整路径

编号与本阶段正文一致。路径相对于配套 RISCV_vdot 工程根目录；图片随本文档目录提供，完整源码、日志和大型 VCD 由配套工程提供。

编号	材料	项目相对路径
1	项目 v0.1 指令规格	docs/vdot-spec-v0.1.md
2	vdot.h 源码	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU/src/isa/riscv64/instr/rvv/vdot.h
3	NEMU 修改快照	artifacts/vdot-integration-2026-10-24/delivery/source-snapshot/NEMU
4	接口核对记录	artifacts/scnet-abi-2026-10-24/abi-review.json
5	交付与复现记录	reports/stage3/delivery.md
6	原始 API 结果	artifacts/vdot-integration-2026-10-24/nemu-api-151012/result.json
7	修复回归	artifacts/vdot-mtval-fix-2026-10-24/server-xYGYlZ/review.json
8	流水回归	artifacts/vdot-pipeline-2026-10-24/server-JuGUz6/review.json
9	差分诊断记录	artifacts/vdot-matrix-2026-10-24/server-OMcrpf/diagnosis.md
10	修复说明	artifacts/vdot-mtval-fix-2026-10-24/README.md
11	本项目汇编	benchmarks/rvv-baseline/vdot-rvv-equiv-v1/cases.S
12	本地结果	artifacts/vdot-rvv-equivalence-2026-10-26/local-170237/result.json
13	服务器核验	artifacts/vdot-rvv-equivalence-2026-10-26/server-8nvimb/review.json
14	应用源码	benchmarks/rvv-baseline/vdot-rvv-equiv-v1
15	实际构建运行入口	artifacts/vdot-rvv-equivalence-2026-10-26/server-paste-command.txt
16	归档逐轮表	artifacts/stage3-remote-archive-2026-10-27/server-MIwNSR/complete-samples.md

编号	材料	项目相对路径
17	补测结果	artifacts/vdot-load-reuse-2026-10-30/server-review.json
18	计时程序	benchmarks/rvv-baseline/vdot-timing-pilot-v1/cases.S
19	预检记录	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/review.json
20	原始统计	artifacts/vdot-chain-2026-10-26/server-KiI3xX/raw-statistics.json
21	依赖链程序	benchmarks/rvv-baseline/vdot-chain-v1/cases.S
22	运行记录	artifacts/vdot-chain-2026-10-26/server-KiI3xX/review.json
23	原始统计	artifacts/vdot-throughput-2026-10-26/server-uLgHm6/raw-statistics.json
24	原始统计	artifacts/vdot-e2e-2026-10-27/server-bijzGZ/raw-statistics.json
25	完整计数器路径	artifacts/stage3-closeout-2026-10-27/e2e-read/perf-context.json
26	分段核验	artifacts/vdot-segment-2026-10-29/server-Pw1wQ8/review.json
27	加载实验记录	artifacts/vdot-load-reuse-2026-10-30/load/server-L4TEPC/review.md
28	复用实验记录	artifacts/vdot-load-reuse-2026-10-30/tile/server-x9yIzN/review.md
29	vdot-timing-pilot-v1	benchmarks/rvv-baseline/vdot-timing-pilot-v1
30	运行入口	artifacts/vdot-timing-pilot-2026-10-26/retry-budget/server-paste-command.txt
31	统计	artifacts/vdot-timing-pilot-2026-10-26/server-5vHUp8/raw-statistics.json
32	vdot-chain-v1	benchmarks/rvv-baseline/vdot-chain-v1
33	命令	artifacts/vdot-chain-2026-10-26/server-paste-command.txt
34	vdot-throughput-v1	benchmarks/rvv-baseline/vdot-throughput-v1
35	命令	artifacts/vdot-throughput-2026-10-26/server-paste-command.txt
36	vdot-e2e-v1	benchmarks/rvv-baseline/vdot-e2e-v1
37	命令	artifacts/vdot-e2e-2026-10-27/server-paste-command.txt
38	vdot-segment-v1	benchmarks/rvv-baseline/vdot-segment-v1
39	命令	artifacts/vdot-segment-2026-10-29/server-paste-command.txt
40	方法及源码入口	reports/stage3/load-reuse-plan.md
41	本地原件归档	artifacts/stage3-remote-archive-2026-10-27/server-MIwNSR